

Programação Funcional

Criando tipos em Haskell

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 Introdução
- 2 Estrutura de uma Árvore
- 3 Operações Básicas

Tópicos

- 1 Introdução
- 2 Estrutura de uma Árvore
- 3 Operações Básicas

Tópicos

- 1 Introdução
- 2 Estrutura de uma Árvore
- 3 Operações Básicas

Introdução

O que são Árvores na computação?

- Uma árvore é uma estrutura de dados hierárquica onde cada elemento, chamado *nó*, pode ter filhos. O nó raiz é o ponto de entrada da estrutura.
- As árvores são fundamentais para organizar dados de forma eficiente. Elas podem ser usadas em estruturas:
 - Árvores binárias de pesquisa - cada nó tem no máximo dois filhos ordenados por precedência
 - Árvores de decisão - para classificar dados para aprendizado de máquina
- As árvores também podem representar relações hierárquicas, como em sistemas de arquivos e estruturas genealógicas, o que facilita a estruturação e o acesso às informações complexas.

Introdução

Uma árvore é uma estrutura de dados hierárquica composta por:

- **Raiz:** Representa o nó principal da árvore, que é o nó de entrada e não tem pai. Todas as outras estruturas da árvore derivam da raiz.
- **Nó:** Representa um nó da árvore que contém um valor do tipo *a* e pode ter zero, um ou mais filhos.
- **Filho:** Representa um nó conectado diretamente a outro nó (o pai) e pode ser nulo ou vazio.

Definição de uma árvore em Haskell

Podemos definir a estrutura de uma árvore binária destarte:

```
data Tree a = Leaf a
             | Node a (Tree a) (Tree a)
             | Empty
             deriving(Eq,Show)
```

onde:

- **Leaf**: Representa uma folha na árvore, que é um nó terminal sem filhos.
- **Node**: Representa um nó interno da árvore que contém um valor do tipo *a* e tem dois filhos, que são subárvores (esquerda e direita).
- **Empty**: Representa um nó nulo ou vazio. É uma forma de indicar a ausência de um nó em algum ponto da árvore.

Definição de uma árvore em Haskell

```
data Tree a = Leaf a
            | Node a (Tree a) (Tree a)
            | Empty
            deriving(Eq,Show)
```

- Define um novo tipo de dados chamado *Tree* que é parametrizado por um tipo *a*. Se for *Leaf*, será uma folha com valor do tipo *a* e sem filhos.
- Se for *Node*, ele possuirá três componentes, que são: Um valor do tipo *a*, e uma subárvore à esquerda e uma subárvore à direita.
- *Empty* é um construtor de dados que representa um nó nulo ou vazio. Ele é usado para indicar a ausência de um nó e/ou distinguir claramente entre um nó vazio e uma folha.
- Herda as classes *Eq* e *Show*

Definição de uma árvore em Haskell

```
data Tree a = Leaf a
            | Node a (Tree a) (Tree a)
            | Empty
            deriving(Eq,Show)
```

- Define um novo tipo de dados chamado *Tree* que é parametrizado por um tipo *a*. Se for *Leaf*, será uma folha com valor do tipo *a* e sem filhos.
- Se for *Node*, ele possuirá três componentes, que são: Um valor do tipo *a*, e uma subárvore à esquerda e uma subárvore à direita.
- *Empty* é um construtor de dados que representa um nó nulo ou vazio. Ele é usado para indicar a ausência de um nó e/ou distinguir claramente entre um nó vazio e uma folha.
- Herda as classes *Eq* e *Show*

Definição de uma árvore em Haskell

```
data Tree a = Leaf a
            | Node a (Tree a) (Tree a)
            | Empty
            deriving(Eq,Show)
```

- Define um novo tipo de dados chamado *Tree* que é parametrizado por um tipo *a*. Se for *Leaf*, será uma folha com valor do tipo *a* e sem filhos.
- Se for *Node*, ele possuirá três componentes, que são: Um valor do tipo *a*, e uma subárvore à esquerda e uma subárvore à direita.
- *Empty* é um construtor de dados que representa um nó nulo ou vazio. Ele é usado para indicar a ausência de um nó e/ou distinguir claramente entre um nó vazio e uma folha.
- Herda as classes *Eq* e *Show*

Definição de uma árvore em Haskell

```
data Tree a = Leaf a
            | Node a (Tree a) (Tree a)
            | Empty
            deriving(Eq,Show)
```

- Define um novo tipo de dados chamado *Tree* que é parametrizado por um tipo *a*. Se for *Leaf*, será uma folha com valor do tipo *a* e sem filhos.
- Se for *Node*, ele possuirá três componentes, que são: Um valor do tipo *a*, e uma subárvore à esquerda e uma subárvore à direita.
- *Empty* é um construtor de dados que representa um nó nulo ou vazio. Ele é usado para indicar a ausência de um nó e/ou distinguir claramente entre um nó vazio e uma folha.
- Herda as classes *Eq* e *Show*

Inserção

Exercício proposto

Com base na estrutura de árvore binária já definida, implemente a função de *inserção* para inserir um novo nó na árvore.

Percursos

Exemplo 1- Percurso Em Ordem

```
emOrdem :: Tree a -> [a]
emOrdem Empty = []
emOrdem (Leaf a) = [a]
emOrdem (Node a e d) = emOrdem e ++ [a] ++ emOrdem d
```

Exemplo 2- Percurso Pós-Ordem

```
posOrd :: Tree a -> [a]
posOrd Empty = []
posOrd (Leaf a) = [a]
posOrd (Node a e d) = posOrd e ++ posOrd d ++ [a]
```

- Como ficaria o percurso pré-ordem?

Referências Bibliográficas

- Book [1]



Simon Thompson.

Haskell: the Craft of Functional Programming.

Addison-Wesley, Harlow, England, 1997.

- Rodrigo Geraldo Ribeiro -

<https://github.com/rodrigogribeiro>

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os
tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos
sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a
bibliografia citada no plano de ensino da disciplina para estudar os
tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço aos monitores da disciplina Marcos Vyctor Fonseca Galupo e Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.